

Chapter 08

strings, string_views, Text Files, CSV Files and Regex

Learning Objectives

- Perform various string manipulations, including assignment, concatenation, comparison, obtaining substrings, searching within strings, modifying strings and converting strings to numbers.
- Use `string_views` for lightweight views of contiguous characters.
- Write and read sequential text files.
- Input and output quoted text.
- Perform input from and output to strings in memory.
- Use raw string literals to automatically escape special characters in strings and to create multiline strings.

Introduction

- This chapter discusses additional `std::string` features and introduces `string_views`, `text-file processing`, `CSV-file processing` and `regular expressions`.
- `std::string`
 - We've been using `std::string` objects since Chapter 2.
 - Here, we introduce many more `std::string` manipulations, including assignments, comparisons, extracting substrings, searching for substrings and modifying `std::string` objects.
 - We also discuss converting `std::string` objects to numeric values and vice versa.

Introduction (Cont.)

- `std::string_view`
 - We introduce `string_views`—read-only views of C-strings or `std::string` objects.
 - Like `std::span`, a `string_view` does not own the data it views.
 - You'll see that `string_views` have similar capabilities to `std::strings`, making them appropriate for cases where you do not need modifiable strings.

Introduction (Cont.)

- Text Files and String Stream Processing
 - Data in memory is temporary. Files are used for data persistence—permanent data retention. Computers store files on secondary storage devices, such as flash drives, and frequently today, in the cloud. We explain how to build C++ programs that create, update and process text files. We also show how to output data to and read data from a `std::string` in memory using `ostringstreams` and `istringstreams`.

Introduction (Cont.)

- Objects-Natural Data Science Case Study: CSV Files and the Titanic Disaster Dataset
 - This chapter's first of three Objects-Natural case studies introduces the CSV (comma-separated values) file format.
 - ◆ CSV is popular for datasets used in big data, data analytics and data science, and artificial intelligence applications like natural language processing, machine learning and deep learning.
 - A commonly used dataset for data science beginners is the Titanic disaster dataset.
 - ◆ It lists all the passengers and whether they survived when the ship Titanic struck an iceberg and sank during its first voyage of April 10–15, 1912.
 - ◆ We use a class from the open-source rapidcsv library to create an object that reads the Titanic dataset from a CSV file.
 - ◆ Then, we view some of the data and perform some basic data analytics.

Introduction.

- Objects-Natural Data Science Case Study: Using Regular Expressions to Search Strings for Patterns, Validate Data and Replace Substrings
 - This chapter's third Objects-Natural case study introduces regular expressions, which are particularly crucial in today's data-rich applications for
 - ◆ cleaning and preparing data for analysis,
 - ◆ data mining—such as locating URLs, email addresses and phone numbers in large bodies of text—and
 - ◆ transforming data to other formats—such as converting tab-delimited data to comma-delimited data.
 - We'll use regex objects to create regular expressions, then use them with various functions in the <regex> header to match patterns in text.
 - Earlier chapters mentioned the importance of validating user input in industrial-strength code.
 - The `std::string`, string stream and regular expression capabilities presented in this chapter are frequently used to validate data.

string Assignment and Concatenation

- The next program demonstrates various `std::string` assignment and concatenation capabilities.

```
1 // Demonstrating string assignment and concatenation.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     std::string s1{"cat"};
8     std::string s2; // initialized to the empty string
9     std::string s3; // initialized to the empty string
10
11     s2 = s1; // assign s1 to s2
12     s3.assign(s1); // assign s1 to s3
13     std::cout << std::format("s1: {}\ns2: {}\ns3: {}\n\n", s1, s2, s3);
```

string Assignment and Concatenation (Cont.)

- String Assignment

- Lines 7–9 create the string variables s1, s2 and s3.
- Line 11 uses the assignment operator to copy the contents of s1 into s2.

```
1 // Demonstrating string assignment and concatenation.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     std::string s1{"cat"};
8     std::string s2; // initialized to the empty string
9     std::string s3; // initialized to the empty string
10
11     s2 = s1; // assign s1 to s2
12     s3.assign(s1); // assign s1 to s3
13     std::cout << std::format("s1: {}\\ns2: {}\\ns3: {}\\n\\n", s1, s2, s3);
```

string Assignment and Concatenation (Cont.)

- String Assignment

- Line 12 uses member function `assign` to copy `s1`'s contents into `s3`.
 - ◆ This particular version of `assign` is equivalent to using the `=` operator, but `assign` also has many overloads.
 - ◆ For example, one overload copies a specified number of characters, as in `target.assign(source, start, numberOfChars);` where `source` is the string to copy, `start` is the starting index, and `numberOfChars` is the number of characters to copy.

```
11 | s2 = s1; // assign s1 to s2
12 | s3.assign(s1); // assign s1 to s3
13 | std::cout << std::format("s1: {} \ns2: {} \ns3: {} \n\n", s1, s2, s3);
```

```
s1: cat
s2: cat
s3: cat
```

string Assignment and Concatenation (Cont.)

- Accessing String Elements By Index

- Lines 15–16 use the string member function `at` to assign 'r' to `s2` at index 0 (forming "rat") and to assign 'r' to `s3` at index 2 (forming "car").
- You can also use the member function `at` to get the character at a specific index in a string.
 - ◆ As with `std::array` and `std::vector`, a `std::string`'s `at` member function performs range checking and throws an `out_of_range` exception if the index is not within the string's bounds.
 - ◆ The string subscript operator, `[]`, does not check whether the index is in bounds.
 - This is consistent with its use with `std::array` and `std::vector`.
 - ◆ You also can iterate through the characters in a string using range-based for, as in

```
for (char c : s3) {  
    cout << c;  
}
```

which ensures that you do **NOT** access elements outside the string's bounds.

```
15 s2.at(0) = 'r'; // modify s2  
16 s3.at(2) = 'r'; // modify s3  
17 std::cout << std::format("After changes:\ns2: {}\ns3: {}", s2, s3);
```

After changes:
s2: rat
s3: car

string Assignment and Concatenation (Cont.)

- String Concatenation

- Line 20 initializes s4 to the contents of s1, followed by "apult".
- For std::string, the + operator denotes string concatenation.
- Line 21 uses the member function append to concatenate s1 and "acomb".
- Next, line 22 uses the overloaded addition assignment operator, +=, to concatenate s3 and "pet".

```
19     std::cout << "\n\nAfter concatenations:\n";
20     std::string s4{s1 + "apult"}; // concatenation
21     s1.append("acomb"); // create "catacomb"
22     s3 += "pet"; // create "carpet" with overloaded +=
23     std::cout << std::format("s1: {}\ns3: {}\ns4: {}\n", s1, s3, s4);
24
25     // append locations 4 through end of s1 to
26     // create string "comb" (s5 was initially empty)
27     std::string s5; // initialized to the empty string
28     s5.append(s1, 4, s1.size() - 4);
29     std::cout << std::format("s5: {}\n", s5);
```

After concatenations:
s1: catacomb
s3: carpet
s4: catapult
s5: comb

string Assignment and Concatenation.

- String Concatenation

- Then line 28 appends the string "comb" to empty string s5.
 - ◆ The arguments are the std::string to retrieve characters from (s1), the starting index (4) and the number of characters to append (s1.size() - 4).

```
19 std::cout << "\n\nAfter concatenations:\n";
20 std::string s4{s1 + "apult"}; // concatenation
21 s1.append("acomb"); // create "catacomb"
22 s3 += "pet"; // create "carpet" with overloaded +=
23 std::cout << std::format("s1: {}\ns3: {}\ns4: {}\n", s1, s3, s4);
24
25 // append locations 4 through end of s1 to
26 // create string "comb" (s5 was initially empty)
27 std::string s5; // initialized to the empty string
28 s5.append(s1, 4, s1.size() - 4);
29 std::cout << std::format("s5: {}\n", s5);
```

After concatenations:

- s1: catacomb
- s3: carpet
- s4: catapult
- s5: comb

Comparing strings

- `std::string` provides member functions for comparing strings.
- In the next example, we call function `displayResult` (lines 6–16) to display each comparison's result.

```
1 // Comparing strings.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 void displayResult(const std::string& s, int result) {
7     if (result == 0) {
8         std::cout << std::format("{} == 0\n", s);
9     }
10    else if (result > 0) {
11        std::cout << std::format("{} > 0\n", s);
12    }
13    else { // result < 0
14        std::cout << std::format("{} < 0\n", s);
15    }
16 }
```

Comparing strings (Cont.)

- The program declares four strings (lines 19–22) and outputs each (lines 24–25).

```
18  int main() {  
19      const std::string s1{"Testing the comparison functions."};  
20      const std::string s2{"Hello"};  
21      const std::string s3{"stinger"};  
22      const std::string s4{s2}; // "Hello"  
23  
24      std::cout << std::format("s1: {}\ns2: {}\ns3: {}\ns4: {}\n\n",  
25                              s1, s2, s3, s4);
```

```
s1: Testing the comparison functions.  
s2: Hello  
s3: stinger  
s4: Hello
```

Comparing strings (Cont.)

- Comparing Strings with the Relational and Equality Operators
 - `std::string` objects may be compared to one another or C-strings with the relational and equality operators.
 - ◆ Comparisons are performed lexicographically—that is, based on the integer values of each character.
 - ◆ For example, 'A' has the value 65 and 'a' has the value 97, so "Apple" would be considered less than "apple", even though they are the same word.

Comparing strings (Cont.)

- Comparing Strings with the Relational and Equality Operators

- Line 28 tests whether s1 is greater than s4 using the overloaded > operator.
 - ◆ In this case, s1 starts with a capital T, and s4 starts with a capital H. So, s1 is greater than s4 because T (84) has a higher numeric value than H (72).

```
27 // comparing s1 and s4
28 if (s1 > s4) {
29     std::cout << "s1 > s4\n";
30 }
```

s1 > s4

Comparing strings (Cont.)

- Comparing Strings with Member Function compare
 - Line 33 compares s1 to s2 using std::string member function compare.
 - ◆ This function returns 0 if the strings are equal,
 - ◆ a positive number if s1 is lexicographically greater than s2
 - ◆ or a negative number if s1 is lexicographically less than s2.
 - ◆ Because a string starting with 'T' is considered lexicographically greater than a string starting with 'H', the result is a value greater than 0, as confirmed by the output.

```
32 // comparing s1 and s2
33 displayResult("s1.compare(s2)", s1.compare(s2));
```

```
s1.compare(s2) > 0
```

Comparing strings (Cont.)

- Comparing Strings with Member Function compare
 - The compare call in line 37 compares portions of s1 and s3 using a compare overload.
 - ◆ The first two arguments (2 and 5) specify the starting index and length of the portion of s1 ("sting") to compare with s3.
 - ◆ The third argument is the comparison string.
 - ◆ The last two arguments (0 and 5) are the starting index and length of the portion of s3 to compare (also "sting").
 - ◆ The two pieces being compared are identical, so compare returns 0, as confirmed in the output.

```
35 // comparing s1 (elements 2-6) and s3 (elements 0-4)
36 displayResult("s1.compare(2, 5, s3, 0, 5)",
37             s1.compare(2, 5, s3, 0, 5));
```

```
s1.compare(2, 5, s3, 0, 5) == 0
```

Comparing strings.

- Comparing Strings with Member Function compare
 - Line 41 uses another compare overload to compare s4 and s2.
 - ◆ The first two arguments are the starting index and length,
 - ◆ and the last argument is the comparison string.
 - ◆ The pieces of s4 and s2 being compared are identical, so compare returns 0.
 - Line 44 compares the first 3 characters in s2 to s4.
 - ◆ Because "Hel" begins with the same first three letters as "Hello" but has fewer letters overall, "Hel" is considered less than "Hello" and compare returns a value less than zero.

```
39 // comparing s2 and s4
40 displayResult("s4.compare(0, s2.size(), s2)",
41             s4.compare(0, s2.size(), s2));
42
43 // comparing s2 and s4
44 displayResult("s2.compare(0, 3, s4)", s2.compare(0, 3, s4));
45 }
```

`s4.compare(0, s2.size(), s2) == 0`
`s2.compare(0, 3, s4) < 0`

Substrings.

- `std::string`'s member function `substr` returns a substring from a string.
- The result is a new string object with contents copied from the source string.
- Line 7 uses member function `substr` to get a substring from `s` starting at index 3 and consisting of 4 characters.

```
1 // Demonstrating string member function substr.
2 #include <iostream>
3 #include <string>
4
5 int main() {
6     const std::string s{"airplane"};
7     std::cout << s.substr(3, 4) << '\n'; // retrieve substring "plan"
8 }
```

plan

Swapping strings

- `std::string`'s `swap` member function for swapping the contents of two strings. The next program swaps the values of `s1` and `s2`, which are not required to have the same length.

```
1 // Using the swap function to swap two strings.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     std::string s1{"one"};
8     std::string s2{"two"};
9
10    std::cout << std::format("Before swap:\ns1: {}; s2: {}\n\n", s1, s2);
11    s1.swap(s2); // swap strings
12    std::cout << std::format("After swap:\ns1: {}; s2: {}\n", s1, s2);
13 }
```

Before swap:
s1: one; s2: two

After swap:
s1: two; s2: one

string Characteristics

- `std::string` provides member functions for gathering information about a string's size, capacity, maximum length and other characteristics:
 - A string's size is **the number of characters** currently stored in the string.
 - A string's **capacity** is the number of characters that can be stored in the string **before it must allocate more memory** to store additional characters.
 - ◆ A string performs memory allocation for you behind the scenes.
 - ◆ The capacity of a string is always at least the string's current size, though it can be greater. The exact capacity depends on the implementation.
 - The maximum size is the largest possible size a string can have. If this value is exceeded, a **length_error exception** is thrown.
 - ◆ In practice, the maximum size is so large in most implementations that you will not encounter this exception.

string Characteristics (Cont.)

- The next program demonstrates string member functions for determining these characteristics.
 - Function `printStats` (lines 7–11) receives a string and displays its
 - ◆ capacity (using member function `capacity`),
 - ◆ maximum size (using member function `max_size`),
 - ◆ size (using member function `size`) and
 - ◆ whether the string is empty (using member function `empty`).

```
1 // Printing string characteristics.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 // display string statistics
7 void printStatistics(const std::string& s) {
8     std::cout << std::format(
9         "capacity: {} \nmax size: {} \nsize: {} \nempty: {}",
10        s.capacity(), s.max_size(), s.size(), s.empty());
11 }
```

string Characteristics (Cont.)

- The program declares empty string `string1` (line 14) and passes it to function `printStatistics` (line 17).
 - ◆ This call to `printStatistics` indicates that `string1`'s initial size is 0—it contains no characters.
 - ◆ The maximum sizes of strings are larger than you'll typically need.
 - ◆ Object `string1` is an empty string, so function `empty` returns `true`.

```
13 int main() {  
14     std::string string1; // empty string  
15  
16     std::cout << "Statistics before input:\n";  
17     printStatistics(string1);  
}
```

Statistics before input:
capacity: 15
max size: 9223372036854775807
size: 0
empty: true

string Characteristics (Cont.)

- Line 20 inputs a string (we typed tomato).
- Line 23 calls printStatistics to output updated string1 statistics.
 - ◆ The size is now 6, and string1 is no longer empty.

```
19      std::cout << "\n\nEnter a string: ";
20      std::cin >> string1; // delimited by whitespace
21      std::cout << std::format("The string entered was: {}\n", string1);
22      std::cout << "Statistics after input:\n";
23      printStatistics(string1);
```

```
Enter a string: tomato
The string entered was: tomato
Statistics after input:
capacity: 15
max size: 9223372036854775807
size: 6
empty: false
```

string Characteristics (Cont.)

- Line 26 inputs another string (we typed soup) and stores it in string1, replacing "tomato".
- Line 29 calls printStatistics to output updated string1 statistics.
 - ◆ Note that the length is now 4.

```
25     std::cout << "\n\nEnter a string: ";
26     std::cin >> string1; // delimited by whitespace
27     std::cout << std::format("The string entered was: {}\n", string1);
28     std::cout << "Statistics after input:\n";
29     printStatistics(string1);
```

```
Enter a string: soup
The string entered was: soup
Statistics after input:
capacity: 15
max size: 9223372036854775807
size: 4
empty: false
```

string Characteristics (Cont.)

- Line 32 uses += to concatenate a 46-character string to string1.
- Line 35 calls printStatistics to output updated string1 statistics.
 - ◆ Because string1's capacity was not large enough to accommodate the new string size, the capacity was automatically increased to 63 elements, and string1's size is now 50.
 - ◆ How the capacity grows is implementation-defined.

```
31 // append 46 characters to string1
32 string1 += "1234567890abcdefghijklmnopqrstuvwxyz1234567890";
33 std::cout << std::format("\n\nstring1 is now: {}\n", string1);
34 std::cout << "Statistics after concatenation:\n";
35 printStatistics(string1);
```

```
string1 is now: soup1234567890abcdefghijklmnopqrstuvwxyz1
234567890
Statistics after concatenation:
capacity: 63
max size: 9223372036854775807
size: 50
empty: false
```

string Characteristics.

- Line 37 uses the member function `resize` to increase `string1`'s size by 10 characters.
 - ◆ The additional elements are set to null characters.
 - ◆ The `printStats` output shows that the capacity did not change, but the size is now 60.

```
37 string1.resize(string1.size() + 10); // add 10 elements to string1
38 std::cout << "\n\nStatistics after resizing to add 10 characters:\n";
39 printStatistics(string1);
40 std::cout << '\n';
41 }
```

```
Statistics after resizing to add 10 characters:
capacity: 63
max size: 9223372036854775807
size: 60
empty: false
```

Finding Substrings and Characters in a string

- `std::string` provides member functions for finding substrings and characters in a string.
- The next program demonstrates the find functions.
- String `s` is declared and initialized in line 7.

```
1 // Demonstrating the string find member functions.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     const std::string s{"noon is 12pm; midnight is not"};
8     std::cout << "Original string: " << s;
```

Original string: noon is 12pm; midnight is not

Finding Substrings and Characters in a string (Cont.)

- Member Functions `find` and `rfind`

- Lines 11–12 attempt to find "is" in `s` using member functions `find` and `rfind`, which search from the beginning and end of `s`, respectively.
 - ◆ If "is" is found, the index of the starting location of that string is returned.
 - ◆ If the string is not found, the string find-related functions return the constant `string::npos` to indicate that a substring or character was not found in the string.
 - The rest of the find functions presented in this section return the same type unless otherwise noted.

```
10 // find "is" from the beginning and end of s
11 std::cout << std::format("\ns.find(\"is\"): {} \ns.rfind(\"is\"): {}",
12   s.find("is"), s.rfind("is"));
```

```
s.find("is"): 5
s.rfind("is"): 23
```

```
01234567890123456789012345678
noon is 12pm; midnight is not
```

Finding Substrings and Characters in a string (Cont.)

- Member Function `find_first_of`
 - Line 15 uses the member function `find_first_of` to locate the first occurrence in `s` of any character in "misop".
 - ◆ The search is done from the beginning of `s`.
 - ◆ The character 'o' is found at index 1.

```
14 // find 'o' from beginning
15 auto location{s.find_first_of("misop")};
16 std::cout << std::format("\ns.find_first_of(\"misop\") found {} at {}",
17 s.at(location), location);
```

`s.find_first_of("misop") found o at 1`

01234567890123456789012345678
noon is 12pm; midnight is not

Finding Substrings and Characters in a string (Cont.)

- Member Function `find_last_of`
 - Line 20 uses the member function `find_last_of` to find the last occurrence in `s` of any character in "misop".
 - ◆ The searching is done from the end of `s`.
 - ◆ The character 'o' is found at index 27 of the string.

```
19 // find 'o' from end
20 location = s.find_last_of("misop");
21 std::cout << std::format("\ns.find_last_of(\"misop\") found {} at {}",
22                          s.at(location), location);
```

`s.find_last_of("misop") found o at 27`

01234567890123456789012345678
noon is 12pm; midnight is not

Finding Substrings and Characters in a string (Cont.)

- Member Function `find_first_not_of`
 - Line 25 uses member function `find_first_not_of` to find the first character from the beginning of `s` that is not contained in "noi spm", finding '1' at index 8.

```
24 // find '1' from beginning
25 location = s.find_first_not_of("noi spm");
26 std::cout << std::format(
27     "\ns.find_first_not_of(\"noi spm\") found {} at {}",
28     s.at(location), location);
```

```
s.find_first_not_of("noi spm") found 1 at 8
```

```
01234567890123456789012345678
noon is 12pm; midnight is not
```

Finding Substrings and Characters in a string (Cont.)

- Member Function `find_first_not_of`
 - Line 31 uses the member function `find_first_not_of` to find the first character not contained in "12noi spm". It searches from the beginning of s and finds ';' at index 12.

```
30 // find ';' at location 12
31 location = s.find_first_not_of("12noi spm");
32 std::cout << std::format(
33     "\ns.find_first_not_of(\"12noi spm\") found {} at {}",
34     s.at(location), location);
```

```
s.find_first_not_of("12noi spm") found ; at 12
```

```
01234567890123456789012345678
noon is 12pm; midnight is not
```

Finding Substrings and Characters in a string.

- Member Function `find_first_not_of`
 - Line 37 uses the member function `find_first_not_of` to find the first character not contained in "noon is 12pm; midnight is not".
 - ◆ In this case, the string being searched contains every character specified in the string argument.
 - ◆ Because a character was not found, `string::npos` is returned.

```
36 // search for characters not in "noon is 12pm; midnight is not"
37 location = s.find_first_not_of("noon is 12pm; midnight is not");
38
39 if (location == std::string::npos) {
40     std::cout << std::format("\n{}: not found\n",
41                             "s.find_first_not_of(\"noon is 12pm; midnight is not\")");
42 }
```

```
s.find_first_not_of("noon is 12pm; midnight is not"): not found
```

```
01234567890123456789012345678
noon is 12pm; midnight is not
```

Replacing and Erasing Characters in a string

- The next program demonstrates string member functions for replacing and erasing characters.
 - Lines 8–12 declare and initialize string1.
 - ◆ When string literals are **separated only by whitespace**, the compiler **concatenates** them into a single string literal.
 - ◆ Breaking apart lengthy strings in this manner can make your code **more readable**.

```
1 // Demonstrating string member functions erase and replace.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     // compiler concatenates all parts into one string
8     std::string string1{"The values in any left subtree"
9         "\nare less than the value in the"
10        "\nparent node and the values in"
11        "\nany right subtree are greater"
12        "\nthan the value in the parent node"};
```

Original string:
The values in any left subtree
are less than the value in the
parent node and the values in
any right subtree are greater
than the value in the parent node

Replacing and Erasing Characters in a string (Cont.)

- Line 16 uses string member function `erase` to erase everything `from (and including) the character in position 62 to the end` of `string1`.
 - ◆ Each newline character occupies one character in the string.

```
16 string1.erase(62); // remove from index 62 through end of string1
17 std::cout << std::format("string1 after erase:\n{}\n\n", string1);
```

string1 after erase:
The values in any left subtree
are less than the value in the

Replacing and Erasing Characters in a string (Cont.)

- Lines 19–25 use `find` to locate each occurrence of the space character.
 - ◆ Each space is then replaced with a period by a call to string member function `replace`, which takes three arguments:
 - the `index of the character` in the string at which replacement should begin,
 - the `number of characters` to replace and
 - the `replacement string`.

```
19  size_t position{string1.find(" ")}; // find first space
20
21  // replace all spaces with period
22  while (position != std::string::npos) {
23      string1.replace(position, 1, ".");
24      position = string1.find(" ", position + 1);
25  }
26
27  std::cout << std::format("After first replacement:\n{}\n\n", string1);
```

After first replacement:
The values in any left subtree
are less than the value in the

string1 after erase:
The values in any left subtree
are less than the value in the

Replacing and Erasing Characters in a string (Cont.)

- ◆ Member function `find` returns `string::npos` when the search character is not found.
- ◆ In line 24, we add 1 to position to continue searching from the next character's location.

```
19 size_t position{string1.find(" ")}; // find first space
20
21 // replace all spaces with period
22 while (position != std::string::npos) {
23     string1.replace(position, 1, ".");
24     position = string1.find(" ", position + 1);
25 }
26
27 std::cout << std::format("After first replacement:\n{}\n\n", string1);
```

After first replacement:
The values in any left subtree
are less than the value in the

string1 after erase:
The values in any left subtree
are less than the value in the

Replacing and Erasing Characters in a string (Cont.)

- Lines 29–36 use functions `find` and `replace` to find every period and replace every period and its following character with two semicolons extracted from the replacement string.

```
29 position = string1.find("."); // find first period
30
31 // replace all periods with two semicolons
32 // NOTE: this will overwrite characters
33 while (position != std::string::npos) {
34     string1.replace(position, 2, "xxxxx;yyy", 5, 2);
35     position = string1.find(".", position + 2);
36 }
37
38 std::cout << std::format("After second replacement:\n{}\n", string1);
```

0123456789
xxxxx;yyy

After second replacement:
The;;alues;;n;;ny;;eft;;ubtree
are;;ess;;han;;he;;alue;;n;;he

After first replacement:
The.values.in.any.left.subtree
are.less.than.the.value.in.the

Replacing and Erasing Characters in a string.

- The arguments passed to this version of replace are
 - ◆ the **index of the element** where the replace operation begins,
 - ◆ the **number of characters** to replace,
 - ◆ a **replacement character string** from which a substring is selected to use as replacement characters,
 - ◆ the **element** in the character string where the replacement substring begins and
 - ◆ the **number of characters** in the replacement character string to use.

```
29     position = string1.find("."); // find first period
30
31     // replace all periods with two semicolons
32     // NOTE: this will overwrite characters
33     while (position != std::string::npos) {
34         string1.replace(position, 2, "xxxxx;yyy", 5, 2);
35         position = string1.find(".", position + 2);
36     }
37
38     std::cout << std::format("After second replacement:\n{}\n", string1);
```

0123456789
xxxxx;yyy

After second replacement:
The;;alues;;n;;ny;;eft;;ubtree
are;;ess;;han;;he;;alue;;n;;he

After first replacement:
The.values.in.any.left.subtree
are.less.than.the.value.in.the

Inserting Characters into a string.

- `std::string` provides overloaded member functions for inserting characters into a string.
 - Line 13 uses string member function `insert` to insert "middle " before index 10 of `s1`.
 - Line 14 uses `insert` to insert "xx" **before** `s2`'s index 3.
 - ◆ The last two arguments specify the starting and last element of "xx" to insert.
 - ◆ Using `string::npos` causes the entire string to be inserted.

```
1 // Demonstrating std::string insert member functions.
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     std::string s1("beginning end");
8     std::string s2("12345678");
9
10    std::cout << std::format("Initial strings:\ns1: {}\ns2: {}\n\n",
11                             s1, s2);
12
13    s1.insert(10, "middle "); // insert "middle " at location 10
14    s2.insert(3, "xx", 0, std::string::npos); // insert "xx" at location 3
15
16    std::cout << std::format("Strings after insert:\ns1: {}\ns2: {}\n\n",
17                             s1, s2);
18 }
```

Initial strings:
s1: beginning end
s2: 12345678

Strings after insert:
s1: beginning middle end
s2: 123xx45678

Numeric Conversions

- C++ provides functions that convert numeric values to strings and vice versa.
- Converting Numeric Values to string Objects
 - The `to_string` function (from header `<string>`) returns the string representation of its numeric argument.
 - ◆ The function is overloaded for the fundamental numeric types `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float`, `double` and `long double`.

Numeric Conversions (Cont.)

- Converting string Objects to Numeric Values
 - The `<string>` header provides eight functions to convert string objects to numeric values:

Function	Return type	Function	Return type
<i>Functions that convert to integral types</i>		<i>Functions that convert to floating-point types</i>	
<code>stoi</code>	<code>int</code>	<code>stof</code>	<code>float</code>
<code>stol</code>	<code>long</code>	<code>stod</code>	<code>double</code>
<code>stoul</code>	<code>unsigned long</code>	<code>stold</code>	<code>long double</code>
<code>stoll</code>	<code>long long</code>		
<code>stoull</code>	<code>unsigned long long</code>		

Numeric Conversions (Cont.)

- Converting string Objects to Numeric Values
 - Each function attempts to convert the beginning of its string argument to a numeric value.
 - ◆ If no conversion can be performed, each function throws an `invalid_argument` exception.
 - ◆ If the conversion result is out of range for the function's return type, each function throws an `out_of_range` exception.

Numeric Conversions (Cont.)

- Functions That Convert strings to Integral Types
 - Consider an example of converting a string to an integral value. Assuming the string `string s{"100hello"};` the following statement converts the beginning of the string to the int value 100 and stores that value in `convertedInt`:
`int convertedInt{stoi(s)};`

Numeric Conversions (Cont.)

- Functions That Convert strings to Integral Types
 - Each function that converts a string to an integral type receives three parameters—the last two have default arguments. The parameters are
 - ◆ A string containing the characters to convert.
 - ◆ A pointer to a `size_t` variable. The function uses this pointer to store the index of the first character that was not converted. The default argument is `nullptr`, in which case the function does not store the index.
 - ◆ An `int` that's either 0 or a value in the range 2–36 representing the number's base—the default is base 10. If the base is 0, the function auto-detects the base.
 - So, the preceding statement is equivalent to
`int convertedInt{stoi(s, nullptr, 10)};`

Numeric Conversions (Cont.)

- Functions That Convert strings to Integral Types
 - Given a `size_t` variable named `index`, the statement `int convertedInt{stoi(s, &index, 2)};`
 - ◆ converts the binary number "100" (base 2) to an int (100 in binary is the int value 4) and
 - ◆ stores in `index` the location of the letter "h" (the first character that was not converted).

Numeric Conversions.

- Functions That Convert strings to Floating-Point Types
 - The functions that convert strings to floating-point types each receive two parameters:
 - ◆ A string containing the characters to convert.
 - ◆ A pointer to a `size_t` variable where the function stores the index of the first character that was not converted. The default argument is `nullptr`, in which case the function does not store the index.
 - Consider an example of converting the following string to a floating-point value:
`string s{"123.45hello"};`
The statement
`double convertedDouble{stod(s)};`
converts the beginning of `s` to the double value 123.45 and stores that value in the variable `convertedDouble`.
 - ◆ Again, the second argument is `nullptr` by default.

string_view

- A `string_view` (header `<string_view>`) is a **read-only view** of the contents of a C-string, a string object or a range of characters in a container, such as an array or vector of chars.
 - A `string_view` cannot modify the contents it views.
- Like `std::span`, a `string_view` does NOT own the data it views. It contains
 - a pointer to the first character in a contiguous sequence of characters and
 - a count of the number of characters.
- A `string_view` does NOT update automatically if the contents it views change in size after you initialize the `string_view`.

string_view (Cont.)

- string_views enable many string-style operations on C-strings without the overhead of creating and initializing string objects, which copies the C-string contents.
 - The C++ Core Guidelines state that you **should** prefer string objects if you need to “**own character sequences**”—for example, so you can **modify** a string’s contents.
 - If you need a **read-only view** of a contiguous sequence of characters, the C++ Core Guidelines recommend using a **string_view**.

string_view (Cont.)

- Creating a string_view

- The next program demonstrates several string_view features.
 - ◆ Line 5 includes the header <string_view>.
 - ◆ Line 8 creates the string s1, and line 9 copies s1 into the string s2.
 - ◆ Line 10 uses s1 to initialize a string_view.

```
1 // Demonstrating string_view.
2 #include <format>
3 #include <iostream>
4 #include <string>
5 #include <string_view>
6
7 int main() {
8     std::string s1{"red"};
9     std::string s2{s1};
10    std::string_view v1{s1}; // v1 "sees" the contents of s1
11    std::cout << std::format("s1: {}\ns2: {}\nv1: {}\n\n", s1, s2, v1);
```

```
s1: red
s2: red
v1: red
```

string_view (Cont.)

- string_views “See” Changes to the Characters They View

- A string_view does not own the sequence of characters it views but “sees” any changes to the original characters.
- Line 14 modifies the std::string s1.
- Then line 15 shows s1’s, s2’s and the string_view v1’s contents.
 - ◆ Note that s2 was not modified because it owns a copy of s1’s contents.

```
13 // string_views see changes to the characters they view
14 s1.at(0) = 'R'; // capitalize s1
15 std::cout << std::format("s1: {}\ns2: {}\nv1: {}\n\n", s1, s2, v1);
```

s1: Red
s2: red
v1: Red

string_view (Cont.)

- string_views Can be Compared to std::strings or string_views
 - Like strings, string_views support the relational and equality operators.
 - You also can intermix std::strings and string_views as in line 19's == comparisons.

```
17 // string_views are comparable with strings or string_views
18 std::cout << std::format("s1 == v1: {}\ns2 == v1: {}\n\n",
19     s1 == v1, s2 == v1);
```

```
s1 == v1: true
s2 == v1: false
```

string_view (Cont.)

- string_views Can Remove a Prefix or Suffix
 - You can easily remove a specified number of characters from the beginning or end of a string_view.
 - ◆ These are fast operations for a string_view.
 - ◆ They adjust the character count and, if necessary, move the pointer to string_view's first remaining character.
 - Lines 22–23 call string_view member functions remove_prefix and remove_suffix to remove one character from the beginning and one from the end of v1, respectively.
 - ◆ Note that s1 remains unmodified.

string_view (Cont.)

- string_views Can Remove a Prefix or Suffix
 - Lines 22–23 call string_view member functions remove_prefix and remove_suffix to remove one character from the beginning and one from the end of v1, respectively.
 - ◆ Note that s1 remains unmodified.

```
21 // string_view can remove a prefix or suffix
22 v1.remove_prefix(1); // remove one character from the front
23 v1.remove_suffix(1); // remove one character from the back
24 std::cout << std::format("s1: {}\nv1: {}\n\n", s1, v1);
```

```
s1: Red
v1: e
```

string_view (Cont.)

- string_views Are Iterable

- Line 27 initializes a string_view from a C-string.
- Like strings, string_views are iterable, so you can use them with the range-based for statement, as in lines 29–31.

```
26 // string_views are iterable
27 std::string_view v2{"C-string"};
28 std::cout << "The characters in v2 are: ";
29 for (char c : v2) {
30     std::cout << c << " ";
31 }
```

The characters in v2 are: C - s t r i n g

string_view.

- string_views Enable Various String Operations on C-Strings

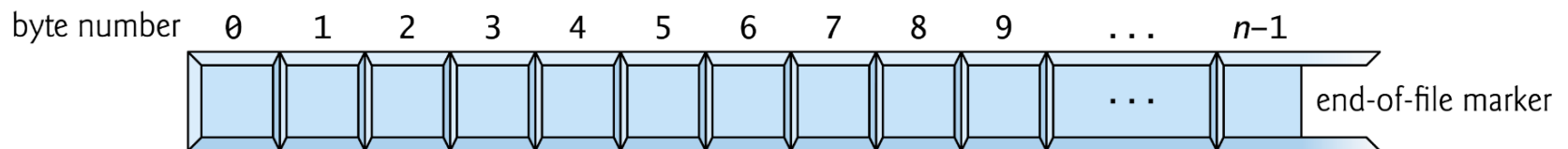
- Many string member functions that do not modify a string's contents also are defined for string_views.
- For example,
 - ◆ line 34 calls `size` to determine how many characters the string_view can “see,”
 - ◆ line 35 calls `find` to get the index of '-' in the string_view and
 - ◆ lines 36–37 use the new C++20 `starts_with` function to determine whether the string_view starts with 'C'.

```
33 // string_views enable various string operations on C-Strings
34 std::cout << std::format("\n\nv2.size(): {}\n", v2.size());
35 std::cout << std::format("v2.find('-'): {}\n", v2.find('-'));
36 std::cout << std::format("v2.starts_with('C'): {}\n",
37     v2.starts_with('C'));
38 }
```

```
v2.size(): 8
v2.find('-'): 1
v2.starts_with('C'): true
```

Files and Streams

- C++ views each file simply as a sequence of bytes.
- Each file ends either with an **end-of-file** marker or at a specific byte number recorded in an operating-system-maintained administrative data structure.



Files and Streams.

- When a file is opened, an object is created, and a stream is associated with the object.
- The objects `cin`, `cout`, `cerr` and `clog` are created for you in the header `<iostream>`.
- The streams associated with these objects provide communication channels between a program and a particular file or device.
 - The `cin` object (standard input stream object) enables a program to input data from the keyboard or other devices.
 - The `cout` object (standard output stream object) enables a program to output data to the screen or other devices.
 - The `cerr` and `clog` standard error stream objects enable a program to output error messages to the screen or other devices.
 - ◆ Messages written to `cerr` are output **immediately**.
 - ◆ In contrast, messages written to `clog` are stored in a memory object called a **buffer**.
 - ◆ When the buffer is **full**, its contents are written to the standard error stream.

Creating a Sequential File

- C++ imposes no structure on files.
 - Thus, a concept like that of a record containing related data items does not exist.
- You structure files to meet your application's requirements.
- The following example shows how to impose a simple record structure on a file.

Creating a Sequential File (Cont.)

- The next example creates a sequential file that might be used in an accounts-receivable system to help keep track of the money owed to a company by its credit clients.
 - For each client, the program obtains the client's **account number**, **name** and **balance** (i.e., the amount the client owes the company for goods and services received in the past).
 - The data obtained for each client constitutes a record for that client.

```
1 // Creating a sequential file.
2 #include <cstdlib> // exit function prototype
3 #include <format>
4 #include <fstream> // contains file stream processing types
5 #include <iostream>
6 #include <string>
```

Creating a Sequential File (Cont.)

- Opening a File

- The program writes data to a file, so we open the file for output by creating an `ofstream` object (line 10) and initializing it with the filename and the file-open mode.
 - ◆ The file-open mode `ios::out` is the default for an `ofstream` object, so the second argument in line 10 is not required.
 - ◆ Use caution when opening an existing file for output (`ios::out`).
 - Existing files opened with mode `ios::out` are truncated—all data in the file is discarded without warning.
 - If the file does not exist, the `ofstream` object creates the file.

```
8  int main() {  
9      // ofstream opens the file  
10     if (std::ofstream output{"clients.txt", std::ios::out}) {
```

Creating a Sequential File (Cont.)

- Opening a File

- Line 10 creates the `ofstream` object **output** associated with the file **clients.txt** and opens it for output.
 - ◆ We did not specify a **path** to the file on disk, so it's placed in the same folder as the program's executable file.
 - ◆ In legacy C++ code, the filename was specified as a pointer-based string.
 - ◆ Modern C++ allows you to specify the filename as a string object.
 - ◆ The `<filesystem>` header provides features for manipulating files and folders.
 - You may also specify the file to open as a `filesystem::path` object.

```
8  int main() {  
9      // ofstream opens the file  
10     if (std::ofstream output{"clients.txt", std::ios::out}) {
```

Creating a Sequential File (Cont.)

- Opening a File

- Below are the file-open modes. These modes can be combined by separating them with the | operator:
 - ◆ `ios::app`—Append all output to the end of the file without modifying any data already in the file.
 - ◆ `ios::ate`—Open a file for output and move to the end of the file. Used to append data to a file. Data can be written anywhere in the file.
 - ◆ `ios::in`—Open a file for input.
 - ◆ `ios::out`—Open a file for output.
 - ◆ `ios::trunc`—Discard the file's contents. This is the default action for `ios::out`.
 - ◆ `ios::binary`—Open a file for binary (i.e., non-text) input or output.

Creating a Sequential File (Cont.)

- Testing Whether a File Was Opened Successfully
 - After creating an ofstream object and attempting to open the file, the if statement uses the file object output as a condition (line 10) to determine whether the open operation succeeded.
 - For a file object, the overloaded **operator bool** implicitly evaluates the file object to true if the file opened successfully or false otherwise.
 - Some possible reasons opening a file might fail are
 - ◆ attempting to open a nonexistent file for reading,
 - ◆ attempting to open a file for reading or writing in a directory that you don't have permission to access and
 - ◆ opening a file for writing when no secondary storage space is available.

```
8 int main() {  
9     // ofstream opens the file  
10    if (std::ofstream output{"clients.txt", std::ios::out}) {
```

Creating a Sequential File (Cont.)

- Testing Whether a File Was Opened Successfully
 - If there was an unsuccessful attempt to open the file, line 25 outputs an error message, and line 26 invokes function `exit` (from `<cstdlib>`) to terminate the program—the argument is returned to the environment that invoked the program.
 - ◆ Passing `EXIT_SUCCESS` (from `<cstdlib>`) to `exit` indicates that the program terminated normally;
 - ◆ passing any other value (e.g., `EXIT_FAILURE`) indicates that the program terminated due to an error.

```
24  
25  else {
26      std::cerr << "File could not be opened\n";
27      std::exit(EXIT_FAILURE);
}
```

Creating a Sequential File (Cont.)

- Processing Data

- If line 10 opens the file successfully, the program begins processing data.
- Lines 11–12 prompt the user to enter the various fields for each record or the end-of-file indicator if data entry is complete.
 - ◆ To enter the end-of-file indicator on Linux or macOS, type <Ctrl-d> on a line by itself.
 - ◆ On Microsoft Windows, type <Ctrl-z>, then press Enter.

```
11         std::cout << "Enter the account, name, and balance.\n"
12             << "Enter end-of-file to end input.\n? ";
13
14         int account;
15         std::string name;
16         double balance;
```

Creating a Sequential File (Cont.)

- Processing Data

- The while statement's condition (line 19) implicitly invokes the `operator bool` function on `cin`.
 - ◆ The condition remains true as long as each input operation with `cin` is successful.
 - ◆ Entering the end-of-file indicator causes the `operator bool` member function to return false.
 - ◆ You also can call the member function `eof` on the input object to determine whether the end-of-file indicator has been entered.

```
18 // read account, name and balance from cin, then place in file
19 while (std::cin >> account >> name >> balance) {
20     output << std::format("{} {} {}\n", account, name, balance);
21     std::cout << "? ";
22 }
23 }
```

Creating a Sequential File (Cont.)

- Processing Data

- Line 19 extracts each set of data into the variables **account**, **name** and **balance**, and determines whether the end-of-file indicator has been entered.
 - ◆ When end-of-file is encountered (that is, when the user enters the end-of-file key combination) or an input operation fails, the operator `bool` returns false, and the while statement terminates.

```
18 // read account, name and balance from cin, then place in file
19 while (std::cin >> account >> name >> balance) {
20     output << std::format("{} {} {}\n", account, name, balance);
21     std::cout << "? ";
22 }
23 }
```

Creating a Sequential File (Cont.)

- Processing Data

- Line 20 writes a set of data to the file **clients.txt**, using the **stream insertion operator** `<<` and the output object we associated with the file at the beginning of the program.
 - ◆ The data may be retrieved by a program designed to read the file.
 - ◆ The file created is simply a text file that can be viewed by any text editor.

```
18 // read account, name and balance from cin, then place in file
19 while (std::cin >> account >> name >> balance) {
20     output << std::format("{} {} {}\n", account, name, balance);
21     std::cout << "? ";
22 }
23 }
```

```
Enter the account, name, and balance.
Enter end-of-file to end input.
? 100 Devi 24.98
? 200 Taylor 345.67
? 300 Huber 0.00
? 400 Karimov -42.16
? 500 Sosa 224.62
? ^Z
```

Creating a Sequential File.

- Closing a File

- Once the user enters the end-of-file indicator, the while loop terminates.
- At this point, we reach the if statement's closing brace, so the output object goes out of scope, which **automatically closes** the file.
- You should always close a file as soon as it's no longer needed in a program.

- Sample Execution

- In the sample execution, we entered information for five accounts, then signaled that data entry was complete by entering the end-of-file indicator (^Z is displayed for Microsoft Windows).
- This dialog window does not show how the data records appear in the file.
- The next part shows how to create a program that reads this file and prints its contents.

Reading Data from a Sequential File

- The previous program created a sequential file.
- The next program reads data sequentially from the file clients.txt and displays the records.

```
1 // Reading and printing a sequential file.
2 #include <cstdlib>
3 #include <format>
4 #include <fstream> // file stream
5 #include <iostream>
6 #include <string>
7
8 int main() {
9     // ifstream opens the file
10    if (std::ifstream input{"clients.txt", std::ios::in}) {
11        std::cout << std::format("{:<10}{:<13}{:>7}\n",
12                                "Account", "Name", "Balance");
13
14        int account;
15        std::string name;
16        double balance;
```

Reading Data from a Sequential File (Cont.)

- Creating an ifstream object opens a file for input.
 - An ifstream is initialized with a filename and file-open mode.
 - Line 10 creates an ifstream object called input that opens the clients.txt file for reading.
 - If a file's contents should not be modified, use `ios::in` to open it only for input to prevent unintentional modification of the file's contents.

```
8  int main() {  
9      // ifstream opens the file  
10     if (std::ifstream input{"clients.txt", std::ios::in}) {  
11         std::cout << std::format("{:<10}{:<13}{:>7}\n",  
12             "Account", "Name", "Balance");  
13     }  
14     int account;  
15     std::string name;  
16     double balance;
```

Reading Data from a Sequential File (Cont.)

- Opening a File for Input

- Objects of class `ifstream` are opened for input by default, so you can omit `ios::in` when you create the `ifstream`, as in `std::ifstream input{"clients.txt"};`
- An `ifstream` object can be created without opening a file—you can attach one to it later.
- Before attempting to retrieve data from the file, line 10 uses the input object as a condition to determine whether the file was opened successfully.
 - ◆ The overloaded operator `bool` returns a `true` if the file was opened; otherwise, it returns `false`.

```
9 // ifstream opens the file
10 if (std::ifstream input{"clients.txt", std::ios::in}) {
```

Reading Data from a Sequential File (Cont.)

- Reading from the File

- Line 19 reads a set of data (i.e., a record) from the file.
- After line 19 executes the first time, `account` has the value 100, `name` has the value "Devi" and `balance` has the value 24.98.
- Each time line 19 executes, it reads another record into the variables `account`, `name` and `balance`.

```
18  
19  
20  
21  
22
```

```
// display each record in file  
while (input >> account >> name >> balance) {  
    std::cout << std::format("{:<10}{:<13}{:>7.2f}\n",  
        account, name, balance);  
}
```

Reading Data from a Sequential File (Cont.)

- Reading from the File

- Lines 20–21 display each record.
- When the end of the file is reached, the implicit call to operator `bool` in the `while` condition returns false, the `ifstream` object goes out of scope at line 23 (which automatically closes the file), and the program terminates.

```
18  
19  
20  
21  
22
```

```
// display each record in file  
while (input >> account >> name >> balance) {  
    std::cout << std::format("{:<10}{:<13}{:>7.2f}\n",  
        account, name, balance);  
}
```

Account	Name	Balance
100	Devi	24.98
200	Taylor	345.67
300	Huber	0.00
400	Karimov	-42.16
500	Sosa	224.62

Reading Data from a Sequential File (Cont.)

- File-Position Pointers

- Programs often read sequentially from the beginning of a file and read all the data consecutively until the desired data is found.
- It might be necessary to process the file sequentially several times (from the beginning) during the execution of a program.
- `istream` and `ostream` provide member functions `seekg` (“seek get”) and `seekp` (“seek put”) to reposition the file-position pointer.
 - ◆ This represents the byte number of the next byte in the file to be read or written.
- Each `istream` object has a `get` pointer, which indicates the byte number in the file from which the next input is to occur.

Reading Data from a Sequential File (Cont.)

- File-Position Pointers

- Each ostream object has a put pointer, which indicates the byte number in the file at which the next output should be placed. The statement
`input.seekg(0);`
repositions the get file-position pointer to the beginning of the file (location 0) attached to input.
 - ◆ The argument to `seekg` is an integer.
 - ◆ If the end-of-file indicator has been set, you'd also need to execute
`input.clear();`
to re-enable reading from the stream.

Reading Data from a Sequential File (Cont.)

- File-Position Pointers

- An optional second argument indicates the seek direction:
 - ◆ `ios::beg` (the default) for positioning relative to the beginning of a stream,
 - ◆ `ios::cur` for positioning relative to the current position in a stream or
 - ◆ `ios::end` for positioning backward relative to the end of a stream.
- When seeking from the beginning of a file, the file-position pointer is an integer value that specifies a location as a number of bytes from the file's starting location.
 - ◆ This is also referred to as the offset from the beginning of the file.

Reading Data from a Sequential File (Cont.)

- File-Position Pointers

- Some examples of moving the get file-position pointer are
// position to the nth byte of fileObject (assumes ios::beg)
`fileObject.seekg(n);`

// position n bytes forward from the current position in
fileObject
`fileObject.seekg(n, ios::cur);`

// position n bytes back from end of fileObject
`fileObject.seekg(n, ios::end);`

// position at end of fileObject
`fileObject.seekg(0, ios::end);`

Reading Data from a Sequential File.

- File-Position Pointers

- The same operations can be performed using ostream member function seekp.
- Member functions tellg and tellp return the current locations of the get and put pointers, respectively.
- The following statement assigns the get file-position pointer value to the variable location of type `std::istream::pos_type`:
`auto location{fileObject.tellg()};`

Reading and Writing Quoted Text

- any text files contain quoted text, such as "C++ How to Program". For example, in files representing HTML5 webpages, attribute values are enclosed in quotes. If you're building a web browser to display the contents of such a webpage, you must be able to read those quoted strings and remove the quotes.
- Suppose you need to read from a text file, as you did in previous example, but with each account's data formatted as follows:
100 "Silvio Flores" 24.98
- Recall that the stream extraction operator >> treats whitespace as a delimiter. So, if we read the preceding data using the expression:
input >> account >> name >> balance
 - the first stream extraction reads 100 into the int variable account, and
 - the second reads only "Silvio into the string variable name.
 - The opening double quote would be part of the string in name.
 - The third stream extraction fails while attempting to read a value for the double variable balance because the next token (i.e., piece of data) in the input stream—"Flores"—is not a double.

Reading and Writing Quoted Text (Cont.)

- Reading Quoted Text

- The stream manipulator `quoted` (header `<iomanip>`) can read quoted text from a stream. It includes any whitespace characters in the quoted text and discards the double-quote delimiters.
- For example, assuming the data
100 "Silvio Flores" 24.98
- The expression
`input >> account >> std::quoted(name) >> balance`
 - ◆ reads 100 into `account`,
 - ◆ reads Silvio Flores as one string into `name` and
 - ◆ reads 24.98 into `balance`.
 - ◆ If the quoted data contains `\` or `\\` escape sequences, each is read and stored in the string as `"` or `\`, respectively.

Reading and Writing Quoted Text.

- Writing Quoted Text
 - Similarly, you can write quoted text to a stream.
 - For example, if name contains Silvio Flores, the statement `ostream << std::quoted(name);` writes to the ostream `"Silvio Flores"`
 - ◆ If the string contains a " or \, it will be displayed as \" or \\\.

Updating Sequential Files

- Data that is formatted and written to a sequential file, cannot be modified in place without the risk of destroying other data in the file.
 - For example, if the name “Devi” needs to be changed to “Manirakiza,” the old name cannot be overwritten without corrupting the file.
 - The record for Devi was written to the file as
300 Devi 0.00
 - If this record were rewritten beginning at the same location in the file using the longer name, the record would be
300 Manirakiza 0.00
 - The new record contains six more characters than the original.
 - Any characters after “i” in “Manirakiza” would overwrite the space separating the name from 0.00, the value 0.0 and the beginning of the next sequential record in the file.
- The problem with the formatted input/output model using the stream insertion operator << and the stream extraction operator >> is that fields—and hence records—can vary in size.
 - For example, values 7, 14, , 2074, and 27383 are all ints, which store the same number of “raw data” bytes internally.
 - However, these integers become different-sized fields, depending on their actual values, when output as formatted text (character sequences). Therefore, the formatted input/output model usually is not used to update records in place.

Updating Sequential Files.

- Such updating can be done with sequential files, but awkwardly.
- For example, to make the preceding name change in a sequential file, we could:
 - copy the records before 300 Devi 0.00 to a new file,
 - write the updated record to the new file, then
 - write the records after 300 Devi 0.00 to the new file.
- Then we could delete the old file and rename the new one.
 - This requires processing every record in the file to update one record.
 - If many records are being updated in one pass of the file, though, this technique can be acceptable.

String Stream Processing

- In addition to standard stream I/O and file stream I/O, C++ includes capabilities for inputting from and outputting to strings in memory.
- These capabilities often are referred to as in-memory I/O or string stream processing.
 - You can read from a string with `istringstream` and
 - write to a string with `ostringstream`,
 - both from the header `<sstream>`.
- Class templates `istringstream` and `ostringstream` provide the same functionality as classes `istream` and `ostream`, plus other member functions specific to in-memory formatting.
 - An `ostringstream` object uses a string object to store the output data. Its `str` member function returns a copy of that string.

String Stream Processing (Cont.)

- One application of string stream processing is data validation.
 - A program can read an entire line at a time from the input stream into a string.
 - Next, a validation routine can scrutinize the string's contents and correct (or repair) the data if necessary.
 - Then the program can input from the string, knowing that the input data is in the proper format.
- To assist with data validation, C++ provides powerful pattern-matching regular expression capabilities.
 - For instance, in a program requiring a U.S. format telephone number (e.g., (800) 555-1212), you can use a regular expression to confirm that a string matches that format.
 - Many websites provide regular expressions for validating e-mail addresses, URLs, phone numbers, addresses and other popular kinds of data.

String Stream Processing (Cont.)

- Demonstrating ostreamstringstream
 - The next program demonstrate the capabilities of string stream.

```
1 // Using an ostreamstringstream object.
2 #include <iostream>
3 #include <sstream> // header for string stream processing
4 #include <string>
5
6 int main() {
7     std::ostreamstringstream output; // create ostreamstringstream object
8
9     const std::string string1{"Output of several data types "};
10    const std::string string2{"to an ostreamstringstream object:"};
11    const std::string string3{"\ndouble: "};
12    const std::string string4{"\n    int: "};
13
14    constexpr double d{123.4567};
15    constexpr int i{22};
```

String Stream Processing (Cont.)

- Demonstrating ostringstream

- Line 18 outputs string string1, string string2, string string3, double d, string string4 and int i—all to output in memory.
- Line 21 displays output.str(), which returns the string created by output in line 18.
- Line 24 appends more data to the string in memory by simply issuing another stream insertion operation to output, then line 25 displays the updated contents.

```
17 // output strings, double and int to ostringstream
18 output << string1 << string2 << string3 << d << string4 << i;
19
20 // call str to obtain string contents of the ostringstream
21 std::cout << "output contains:\n" << output.str();
22
23 // add additional characters and call str to output string
24 output << "\nmore characters added";
25 std::cout << "\n\noutput now contains:\n" << output.str() << '\n';
26 }
```

String Stream Processing (Cont.)

- Demonstrating istreamstringstream
 - An istreamstringstream object inputs data from a string in memory.
 - Data is stored in an istreamstringstream object as characters.
 - Input from the istreamstringstream object works identically to input from any file.
 - The end of the string is interpreted by the istreamstringstream object as end-of-file.

String Stream Processing (Cont.)

- Demonstrating istringstream

- The next program demonstrates input from an istringstream object.
 - ◆ Line 8 creates string inputString, which consists of characters representing two strings ("Atonia" and "test"), an int (123), a double (4.7) and a char ('A').
 - ◆ Line 9 creates the istringstream object input and initializes it to read from inputString.

```
1 // Demonstrating input from an istringstream object.
2 #include <format>
3 #include <iostream>
4 #include <sstream>
5 #include <string>
6
7 int main() {
8     const std::string inputString{"Atonia test 123 4.7 A"};
9     std::istringstream input{inputString};
10    std::string s1;
11    std::string s2;
12    int i;
13    double d;
14    char c;
```

String Stream Processing (Cont.)

- Demonstrating `istringstream`

- The data items in `inputString` are read into variables `s1`, `s2`, `i`, `d` and `c` in line 16, then displayed in lines 18–19.

```
16      input >> s1 >> s2 >> i >> d >> c;  
17  
18      std::cout << "Items extracted from the istringstream object:\n"  
19      << std::format("{}\n{}\n{}\n{}\n{}\n", s1, s2, i, d, c);
```

```
Items extracted from the istringstream object:  
Atonia  
Test  
123  
4.7  
A
```

String Stream Processing.

- Demonstrating istreamstringstream

- Next, we attempt to read from input again in line 22, but the operation fails because there is no more data in inputString to read.
- So the input object evaluates to false, and the else part of the if...else statement executes.

```
21 // attempt to read from empty stream
22 if (long value; input >> value) {
23     std::cout << std::format("\nlong value is: {}\n", value);
24 }
25 else {
26     std::cout << std::format("\ninput is empty\n");
27 }
28 }
```

input is empty

Raw String Literals

- Recall that backslash characters in strings introduce escape sequences—like `\n` for newline and `\t` for tab.
- If you wish to include a backslash in a string, you must use two backslash characters `\\`, making some strings difficult to read.
- For example, Microsoft Windows uses backslashes to separate folder names when specifying a file's location. To represent a file's location on Windows, you might write
`std::string windowsPath{"C:\\MyFolder\\MySubFolder\\MyFile.txt"};`
- For such cases, raw string literals are more convenient. They have the format:
`R"(rawCharacters)"`
 - The [parentheses are required](#) around the rawCharacters that compose the raw string literal.
 - The compiler automatically inserts backslashes as necessary in a raw string literal to properly escape special characters like double quotes (`"`), backslashes (`\`), etc.
- Using a raw string literal, we can write the preceding string as:
`std::string windowsPath{R"(C:\MyFolder\MySubFolder\MyFile.txt)"};`

Raw String Literals (Cont.)

- Raw strings can make code more readable, particularly when using the regular expressions.
 - Regular expressions often contain many backslash characters.
- A raw string literal can include **optional delimiters** up to 16 characters long before the left parenthesis, (, and after the right parenthesis,), as in
`R"MYDELIMITER(J.*\d[0-35-9]-\d\d-\d\d)MYDELIMITER"`
 - The optional delimiters must be identical if provided.
 - The optional delimiters are required if the raw string literal might contain one or more right parentheses.
 - Otherwise, the first right parenthesis encountered would be treated as the end of the raw string literal.

Raw String Literals.

- Raw string literals may also include line breaks, in which case the compiler inserts newline characters.
- For example, the raw string literal
R"(multiple lines
of text)"
is treated as the string literal
"multiple lines\nof text"
 - Caution: Any indentation in the second and subsequent lines of the raw string literal is **included** in the raw string literal.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- The CSV (comma-separated values) file format, which uses the .csv file extension, is particularly popular, especially for datasets used in big data, data analytics and data science, and in artificial intelligence applications like machine learning and deep learning. Here, we'll demonstrate reading from a CSV file.
- Datasets
 - There's an enormous variety of free datasets available online. The OpenML machine learning resource site <https://openml.org> contains over 21,000 free datasets in CSV format.
 - Another great source of datasets is: <https://github.com/awesomedata/awesome-public-datasets>

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- `account.csv`

- For our first example, we've included a simple dataset in `accounts.csv`.
- This file contains the account information, but in the following format:
`account,name,balance`
`100,Devi,24.98`
`200,Taylor,345.67`
`300,Huber,0.0`
`400,Karimov,-42.16`
`500,Sosa,224.62`
- The first row of a CSV file typically contains column names.
- Each subsequent row contains one data record representing the values for those columns. This dataset has three columns representing an account, name and balance.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Using rapidcsv to Read the Contents of a CSV File
 - The rapidcsv338 header-only library <https://github.com/d99kris/rapidcsv> provides the class `rapidcsv::Document` that you can use to read and manipulate CSV files.
 - Many other libraries have built-in CSV support.
 - As in earlier examples that use open-source libraries, you'll need to point the compiler at the rapidcsv subfolder's `src` folder so you can include `<rapidcsv.h>`.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Using rapidcsv to Read the Contents of a CSV File
 - Line 8 creates and initializes a rapidcsv::Document object named document.
 - ◆ This statement loads the specified file ("accounts.csv").
 - ◆ Class Document's member functions enable you to work with the CSV data by row, by column or by individual value in a specific row and column.

```
1 // Reading from a CSV file.
2 #include <format>
3 #include <iostream>
4 #include <rapidcsv.h>
5 #include <vector>
6
7 int main() {
8     rapidcsv::Document document{"accounts.csv"}; // loads accounts.csv
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Using rapidcsv to Read the Contents of a CSV File
 - In this example, lines 9–12 get the data using the class's GetColumn template member function.
 - ◆ This function returns the specified column's data as a `std::vector` containing elements of the type you specify in angle brackets.
 - Line 9's call
`document.GetColumn<int>("account")`
returns a `vector<int>` containing the account numbers for every record.
 - Similarly, the calls in lines 10–11 and 12 return a `vector<string>` and a `vector<double>` containing all the records' names and balances, respectively.

```
9      std::vector<int> accounts{document.GetColumn<int>("account")};
10     std::vector<std::string> names{
11         document.GetColumn<std::string>("name")};
12     std::vector<double> balances{document.GetColumn<double>("balance")};
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Using rapidcsv to Read the Contents of a CSV File
 - Lines 14–20 format and display the file's contents to confirm that they were read properly.

```
14  std::cout << std::format(  
15      "{:<10}{:<13}{:>7}\n", "Account", "Name", "Balance");  
16  
17  for (size_t i{0}; i < accounts.size(); ++i) {  
18      std::cout << std::format("{:<10}{:<13}{:>7.2f}\n",  
19          accounts.at(i), names.at(i), balances.at(i));  
20  }
```

Account	Name	Balance
100	Devi	24.98
200	Taylor	345.67
300	Huber	0.00
400	Karimov	-42.16
500	Sosa	224.62

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- **Caution: Commas in CSV Data Fields**

- Be careful when working with strings containing embedded commas, such as the name "Flores, Silvio".
- If this name were accidentally stored as the two strings "Flores" and "Silvio", that CSV record would have four fields, not three.
- Programs that read CSV files typically expect every record to have the same number of fields; otherwise, problems occur.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- **Caution: Missing Commas and Extra Commas in CSV Files**
 - Be careful when preparing and processing CSV files.
 - ◆ For example, suppose your file is composed of records, each with four comma-separated int values, such as
100,85,77,9
 - ◆ If you accidentally omit one of these commas, as in
100,8577,9
then the record has only three fields, one with the invalid value 8577
 - ◆ If you put two adjacent commas where only one is expected, as in
100,85,,77,9
then you have five fields rather than four, and one of the fields erroneously would be empty.
 - Each of these comma-related errors could confuse programs trying to process the record.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Reading and Analyzing the Titanic Disaster Dataset
 - One of the most commonly used datasets for data analytics beginners is the Titanic disaster dataset.
 - It lists all the passengers and whether they survived when the ship Titanic struck an iceberg and sank during its maiden voyage of April 10–15, 1912. We'll load the dataset in next program, view some of its data and perform some basic data analytics.
 - To download the dataset in CSV format, go to <https://www.openml.org/d/40945> and click the CSV download button in the page's upper-right corner.
 - ◆ This downloads the file phpMYEkMI.csv, which we renamed as titanic.csv.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Getting to Know the Data

- In data analytics, you'll often begin by getting to know your data. One way is simply to look at the raw data.
- If you open the `titanic.csv` file in a text editor or spreadsheet application, you'll see that the dataset contains 1,309 rows, each containing 14 columns—often called features in data analytics.
- We'll use only four columns here:
 - ◆ survived: 1 or 0 for yes or no, respectively.
 - ◆ sex: "female" or "male".
 - ◆ age: The passenger's age. Most ages are integers, but some children under 1 year of age have floating-point values, so we'll process this column as double values.
 - ◆ pclass: 1, 2 or 3 for first class, second class or third class, respectively.
- To learn more about the dataset's origins and its other columns, visit <https://biostat.app.vumc.org/wiki/pub/Main/DataSets/titanic3info.txt>

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Missing Data

- Bad data values and missing values can significantly impact data analysis.
- The Titanic dataset is missing the ages of 263 passengers.
 - ◆ These are represented as ? in the CSV file.
 - ◆ In this example, when we produce descriptive statistics for the passengers' ages, we'll filter out and ignore the missing values.
 - Some data scientists advise against any attempts to insert "reasonable values."
 - Instead, they advocate clearly marking missing data and leaving it up to a data analytics package to handle the issue.
 - Others offer strong cautions.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Loading the Dataset

- The next program uses some C++20 ranges library features we introduced.
- Lines 14–16 create and initialize a `rapidcsv::Document` object named `titanic` that loads "titanic.csv".

```
1 // Reading the Titanic dataset from a CSV file, then analyzing it.
2 #include <format>
3 #include <algorithm>
4 #include <cmath>
5 #include <iostream>
6 #include <numeric>
7 #include <ranges>
8 #include <rapidcsv.h>
9 #include <string>
10 #include <vector>
11
12 int main() {
13     // load Titanic dataset; treat missing age values as NaN
14     rapidcsv::Document titanic{"titanic.csv",
15         rapidcsv::LabelParams{}, rapidcsv::SeparatorParams{},
16         rapidcsv::ConverterParams{true}};
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Loading the Dataset

- The second and third arguments in line 15 are two of the **default arguments** used to initialize a Document object when you create it only by specifying the CSV filename.
 - ◆ Recall from our discussion of default function arguments that when an argument is specified explicitly for a given parameter, all prior arguments in the argument list also must be specified explicitly.
 - ◆ We provided the second and third arguments only **so we could specify the fourth argument**.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Loading the Dataset

- The `rapidcsv::LabelParams{}` argument specifies by default that the CSV file's **first row contains the column names**.
- The `rapidcsv::SeparatorParams{}` argument specifies by default that each record's **fields are separated by commas**.
- The fourth argument `rapidcsv::ConverterParams{true}` enables RapidCSV to **convert missing and bad data values in integer columns to 0 and floating-point columns to NaN** (not a number).
 - ◆ This enables us to load all the data in the age column into a `vector<double>`, including the missing values represented by ?.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Loading the Data to Analyze
 - Lines 19–22 use the `rapidcsv::Document`'s `GetColumn` member function to get each column by name.

```
18 // GetColumn returns column's data as a vector of the appropriate type
19 auto survived{titanic.GetColumn<int>("survived")};
20 auto sex{titanic.GetColumn<std::string>("sex")};
21 auto age{titanic.GetColumn<double>("age")};
22 auto pclass{titanic.GetColumn<int>("pclass")};
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Viewing Some Rows in the Titanic Dataset
 - The 1,309 rows each represent one passenger.
 - ◆ According to Wikipedia, there were approximately 1,317 passengers, and 815 of them died.
 - ◆ For large datasets, it's possible to display only small portions of the data.
 - A common practice when getting to know your data is to display a few rows from the beginning and end of the dataset to get a sense of the data.
 - ◆ The code in lines 25–30 displays the first five elements of each column's data:

```
24 // display first 5 rows
25 std::cout << std::format("First five rows:\n{:<10}{:<8}{:<6}}\n",
26 "survived", "sex", "age", "class");
27 for (size_t i{0}; i < 5; ++i) {
28     std::cout << std::format("{:<10}{:<8}{:<6.1f}}\n",
29         survived.at(i), sex.at(i), age.at(i), pclass.at(i));
30 }
```

First five rows:

survived	sex	age	class
1	female	29.0	1
1	male	0.9	1
0	female	2.0	1
0	male	30.0	1
0	female	25.0	1

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Viewing Some Rows in the Titanic Dataset

- ◆ The code in lines 33–39 displays the last five elements of each column's data.
 - To determine the control variable's starting value, line 35 calls the `rapidcsv::Document's GetRowCount` member function.
 - Then line 36 initializes the control variable to five less than the row count.
 - Note the value `nan`, indicating a missing value for one of the age column's rows.

```
32 // display last 5 rows
33 std::cout << std::format("\nLast five rows:\n{:<10}{:<8}{:<6}}\n",
34     "survived", "sex", "age", "class");
35 const auto count{titanic.GetRowCount()};
36 for (size_t i{count - 5}; i < count; ++i) {
37     std::cout << std::format("{:<10}{:<8}{:<6.1f}}\n",
38         survived.at(i), sex.at(i), age.at(i), pclass.at(i));
39 }
```

Last five rows:			
survived	sex	age	class
0	female	14.5	3
0	female	nan	3
0	male	26.5	3
0	male	27.0	3
0	male	29.0	3

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics

- As part of getting to know a dataset, data scientists often use statistics to describe and summarize data.
- Let's calculate several descriptive statistics for the age column, including the number of passengers for which we have ages, and the average, minimum, maximum and median age values.
- Before performing these calculations, we must remove nan values.
- A calculation that includes the value nan produces nan as the calculation's result.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics

- Lines 42–43 use the C++20 ranges filtering techniques from to keep only the values in the age vector that are not nan
 - ◆ Function `isnan` (header `<cmath>`) returns true if the value is nan.
- Next, lines 44–45 create a `vector<double>` named `cleanAge`.
 - ◆ The vector initializes its elements by iterating through the filtered results in the `removeNaN` pipeline.

```
41 // use C++20 ranges to eliminate missing values from age column
42 auto removeNaN{age |
43     std::views::filter([](const auto& x) {return !std::isnan(x);})};
44 std::vector<double> cleanAge{
45     std::begin(removeNaN), std::end(removeNaN)};
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics for the Cleaned Age Column
 - Now, we can calculate the descriptive statistics.
 - Line 48 sorts `cleanAge`, which will help us determine the minimum, maximum and median values.
 - To count the number of people for which we have valid ages, we simply get `cleanAge`'s size (line 49).

```
47 // descriptive statistics for cleaned ages column
48 std::sort(std::begin(cleanAge), std::end(cleanAge));
49 size_t size{cleanAge.size()};
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics for the Cleaned Age Column
 - Lines 50–57 determine the median.
 - ◆ If cleanAge's size is even, the median is the average of the two middle elements (line 53);
 - ◆ otherwise, it's the middle element (line 56).



```
50 double median{};  
51  
52 if (size % 2 == 0) { // find median value for even number of items  
53     median = (cleanAge.at(size / 2 - 1) + cleanAge.at(size / 2)) / 2;  
54 }  
55 else { // find median value for odd number of items  
56     median = cleanAge.at(size / 2);  
57 }
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics for the Cleaned Age Column
 - Lines 59–65 display the descriptive statistics.
 - ◆ Lines 61–62 calculate the average age by using the `accumulate` algorithm to total the ages, then dividing the result by `size`.
 - ◆ The vector is sorted, so lines 63–64 determine the `minimum` and `maximum` values by calling the vector's `front` and `back` member functions to get the vector's first and last elements, respectively.

```
59 std::cout << "\nDescriptive statistics for the age column:\n"  
60 << std::format("Passengers with age data: {}\n", size)  
61 << std::format("Average age: {:.2f}\n", std::accumulate(  
62     std::begin(cleanAge), std::end(cleanAge), 0.0) / size)  
63 << std::format("Minimum age: {:.2f}\n", cleanAge.front())  
64 << std::format("Maximum age: {:.2f}\n", cleanAge.back())  
65 << std::format("Median age: {:.2f}\n", median);
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics for the Cleaned Age Column
 - The average and median are measures of central tendency.
 - ◆ Each is a way to produce a single value that represents a “central” value in a set of values—that is, a value that is, in some sense, typical of the others.
 - ◆ For the 1046 people with valid ages, the average age was 29.88 years old.
 - The youngest passenger (i.e., the minimum) was just over two months old ($0.17 * 12$ is 2.04), and
 - the oldest (i.e., the maximum) was 80.
 - The median age was 28.

```
Descriptive statistics for the age column:  
Passengers with age data: 1046  
Average age: 29.88  
Minimum age: 0.17  
Maximum age: 80.00  
Median age: 28.00
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Determining Passenger Counts By Class
 - Let's calculate each class's number of passengers.
 - Lines 68–72 define a lambda that counts the number of passengers in a particular class.
 - C++20's `std::ranges::count_if` algorithm counts all the elements in its first argument, **for which the lambda in its second argument returns true**.
 - ◆ A benefit of the `std::ranges` algorithms is that you do not need to specify the beginning and end of the container—the algorithms handle this for you, simplifying your code.
 - ◆ The first argument to `count_if` in this example will be the `vector<int>` named `pclass`.

```
67 // passenger counts by class
68 auto countClass{
69     [](const auto& column, const int classNumber) {
70         return std::ranges::count_if(column,
71                                     [classNumber](int x) {return classNumber == x;});
72     }
73 };
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Determining Passenger Counts By Class

- In line 71's lambda

- [classNumber](int x) {return classNumber == x;}

- ◆ the lambda introducer [classNumber] specifies that the countClass lambda's parameter classNumber is used in this lambda's body—this is known as capturing the countClass variable.

- ◆ By default, capturing is done by value, so line 71's lambda captures a copy of classNumber's value.

- We'll say more about capturing lambdas and the std::ranges algorithms in Chapter 14.

```
67 // passenger counts by class
68 auto countClass{
69     [](const auto& column, const int classNumber) {
70         return std::ranges::count_if(column,
71             [classNumber](int x) {return classNumber == x;});
72     }
73 };
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Determining Passenger Counts By Class

- Lines 75–77 define constants for the three passenger classes.
- Lines 78–80 call the countClass lambda for each passenger class, and
- lines 82–84 display the counts.

```
Passenger counts by class:  
1st: 323  
2nd: 277  
3rd: 709
```

```
75 constexpr int firstClass{1};  
76 constexpr int secondClass{2};  
77 constexpr int thirdClass{3};  
78 const auto firstCount{countClass(pclass, firstClass)};  
79 const auto secondCount{countClass(pclass, secondClass)};  
80 const auto thirdCount{countClass(pclass, thirdClass)};  
81  
82 std::cout << "\nPassenger counts by class:\n"  
83 << std::format("1st: {}\n2nd: {}\n3rd: {}\n\n",  
84               firstCount, secondCount, thirdCount);
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics for the Cleaned Age Column
 - Let's say you want to determine some statistics about people who survived.
 - Lines 87–89 define a lambda that counts survivors using C++20's `std::ranges::count_if` algorithm counts.
 - ◆ Recall that the `survived` column contains 1 or 0 to represent survived or died, respectively.
 - ◆ These also are values that C++ can treat as true (1) or false (0), so the lambda in line 88
`[](auto x) {return x;}`
simply returns the column value.
 - If that value is 1, `count_if` counts that element.

```
86 // percentage of people who survived
87 const auto survivorCount{
88     std::ranges::count_if(survived, [](auto x) {return x;})
89 };
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Basic Descriptive Statistics for the Cleaned Age Column
 - To determine how many people died, line 92 simply subtracts survivorCount from the survived vector's size.
 - Line 94 calculates the percentage of people who survived.

```
91  std::cout << std::format("Survived count: {}\nDied count: {}\n",  
92      survivorCount, survived.size() - survivorCount);  
93  std::cout << std::format("Percent who survived: {:.2f}%\n\n",  
94      100.0 * survivorCount / survived.size());
```

```
Survived count: 500  
Died count: 809  
Percent who survived: 38.20%
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Counting By Sex and By Passenger Class the Numbers of People Who Survived
 - Lines 97–117 iterate through the survived column, using its 1 or 0 value as a condition (line 104).

```
96 // count who survived by male/female, 1st/2nd/3rd class
97 int survivingMales{0};
98 int survivingFemales{0};
99 int surviving1st{0};
100 int surviving2nd{0};
101 int surviving3rd{0};
102
103 for (size_t i{0}; i < survived.size(); ++i) {
104     if (survived.at(i)) {
105         sex.at(i) == "female" ? ++survivingFemales : ++survivingMales;
106
107         if (firstClass == pclass.at(i)) {
108             ++surviving1st;
109         }
110         else if (secondClass == pclass.at(i)) {
111             ++surviving2nd;
112         }
113         else { // third class
114             ++surviving3rd;
115         }
116     }
117 }
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Counting By Sex and By Passenger Class the Numbers of People Who Survived
 - For each survivor, we increment counters for the survivor's sex (survivingFemales and survivingMales in line 105) and pclass (surviving1st, surviving2nd and surviving3rd in lines 107–115).
 - We'll use these counts to calculate percentages.

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data

- Calculating Percentages of People Who Survived
 - Lines 120–129 calculate and display the percentages of:
 - ◆ women who survived,
 - ◆ men who survived,
 - ◆ first-class passengers who survived,
 - ◆ second-class passengers who survived, and
 - ◆ third-class passengers who survived.

```
119 // percentages who survived by male/female, 1st/2nd/3rd class
120 std::cout << std::format("Female survivor percentage: {:.2f}%\n",
121     100.0 * survivingFemales / survivorCount)
122 << std::format("Male survivor percentage: {:.2f}%\n\n",
123     100.0 * survivingMales / survivorCount)
124 << std::format("1st class survivor percentage: {:.2f}%\n",
125     100.0 * surviving1st / survivorCount)
126 << std::format("2nd class survivor percentage: {:.2f}%\n",
127     100.0 * surviving2nd / survivorCount)
128 << std::format("3rd class survivor percentage: {:.2f}%\n",
129     100.0 * surviving3rd / survivorCount);
```

Objects-Natural Data Science Case Study: Reading and Analyzing a CSV File Containing Titanic Disaster Data.

- Calculating Percentages of People Who Survived
 - Of the survivors, about two-thirds were women, and
 - first-class passengers survived at a higher rate than the other passenger classes.

Female survivor percentage: 67.80%

Male survivor percentage: 32.20%

1st class survivor percentage: 40.00

%2nd class survivor percentage: 23.80%

3rd class survivor percentage: 36.20%

Objects-Natural Data Science Case Study: Intro to Regular Expressions

- Sometimes you'll need to recognize patterns in text, like phone numbers, e-mail addresses, ZIP codes, webpage addresses, Social Security numbers and more.
- A regular expression string describes a **search pattern** for matching characters in other strings.
- Regular expressions can help you **extract data** from unstructured text, such as social media posts.
- They're also important for ensuring that data is in the proper format before you attempt to process it.

Objects-Natural Data Science Case Study: Intro to Regular Expressions

- Validating Data

- Before working with text data, you'll often use regular expressions to validate it. For example, you might check that:
 - ◆ A U.S. ZIP code consists of five digits (such as 02215) or five digits followed by a hyphen and four more digits (such as 02215-4775).
 - ◆ A string last name contains only letters, spaces, apostrophes and hyphens.
 - ◆ An e-mail address contains only the allowed characters in the allowed order.
 - ◆ A U.S. Social Security number contains three digits, a hyphen, two digits, a hyphen and four digits, and adheres to other rules about the specific numbers used in each group of digits.

Objects-Natural Data Science Case Study: Intro to Regular Expressions

- Validating Data

- You'll rarely need to create regular expressions for common items like these.
- The following free websites and others offer repositories of existing regular expressions that you can copy and use:
 - ◆ <https://regex101.com>
 - ◆ <https://regexpr.com/>
 - ◆ <http://www.regexlib.com>
 - ◆ <https://www.regular-expressions.info>
- Many sites like these also allow you to test regular expressions to determine whether they'll meet your needs.

Objects-Natural Data Science Case Study: Intro to Regular Expressions.

- Other Uses of Regular Expressions

- Regular expressions also are used to:

- ◆ Extract data from text (known as [scraping](#))—for example, locating all URLs in a webpage.
 - ◆ Clean data—for example, removing data that's not required, removing duplicate data, handling incomplete data, fixing typos, ensuring consistent data formats, removing formatting, changing text case, dealing with outliers and more.
 - ◆ Transform data into other formats—for example, reformatting tab-separated or space-separated values into comma-separated values (CSV) for an application that requires data to be in CSV format.

Matching Complete Strings to Patterns

- To use regular expressions, include the header `<regex>`, which provides several classes and functions for recognizing and manipulating regular expressions.
- The next program demonstrates matching entire strings to regular expression patterns

Matching Complete Strings to Patterns (Cont.)

- Matching Literal Characters

- The `<regex>` function `regex_match` returns true if the entire string in its first argument matches the pattern in its second argument.
 - ◆ By default, pattern matching is case-sensitive.
 - ◆ Later, you'll see how to perform case-insensitive matches.

Matching Complete Strings to Patterns (Cont.)

- Matching Literal Characters

- Let's begin by matching literal characters—that is, characters that match themselves.
 - ◆ Line 8 creates a regex object `r1` for the pattern "02215", containing only literal digits that match themselves in the specified order.
 - ◆ Line 11 calls `regex_match` for the strings "02215" and "51220". Each has the same digits, but only "02215" has them in the correct order for a match.

```
1 // Matching entire strings to regular expressions.
2 #include <format>
3 #include <iostream>
4 #include <regex>
5
6 int main() {
7     // fully match a pattern of literal characters
8     std::regex r1{"02215"};
9     std::cout << "Matching against: 02215\n"
10     << std::format("02215: {}; 51220: {}\n\n",
11                   std::regex_match("02215", r1), std::regex_match("51220", r1));
```

Matching against: 02215
02215: true; 51220: false

Matching Complete Strings to Patterns (Cont.)

- Metacharacters, Character Classes and Quantifiers
 - Regular expressions typically contain various special symbols called **metacharacters**:
[] {} () \ * + ^ \$? . |
 - To match any metacharacter as its literal value, precede it by a backslash \.
 - ◆ For example, \\$ matches a dollar sign (\$) and \\ matches a backslash (\).

Matching Complete Strings to Patterns (Cont.)

- Metacharacters, Character Classes and Quantifiers
 - The `\metacharacter` begins each of the predefined character classes, several of which are shown in the following table with the groups of characters they match.

Character class	Matches
<code>\d</code>	Any digit (0–9).
<code>\D</code>	Any character that is not a digit.
<code>\s</code>	Any whitespace character (such as spaces, tabs and newlines).
<code>\S</code>	Any character that is not a whitespace character.
<code>\w</code>	Any word character (also called an alphanumeric character)—that is, any uppercase or lowercase letter, any digit or an underscore
<code>\W</code>	Any character that is not a word character.

Matching Complete Strings to Patterns (Cont.)

- Matching Digits

- Let's validate a five-digit ZIP code. In the regular expression `\d{5}` (created by the raw string literal in line 15),
 - ◆ `\d` is a character class representing a digit (0–9).
 - A character class is a regular expression escape sequence that matches one character.
 - To match more than one, follow the character class with a quantifier: The quantifier `{5}` repeats `\d` five times as if we had written `\d\d\d\d\d` to match five consecutive digits.
 - ◆ Line 18's second call to `regex_match` returns false because "9876" contains only four consecutive digits.

```
13 // fully match five digits
14 std::regex r2(R"(\d{5})");
15 std::cout << R"(Matching against: \d{5})" << "\n"
16 << std::format("02215: {}; 9876: {}\n\n",
17               std::regex_match("02215", r2),
18               std::regex_match("9876", r2));
```

Matching against: \d{5}
02215: true; 9876: false

Matching Complete Strings to Patterns (Cont.)

- Custom Character Classes

- Characters in square brackets, `[]`, define a custom character class that matches a single character. For example,
 - ◆ `[aeiou]` matches a lowercase vowel,
 - ◆ `[A-Z]` matches an uppercase letter,
 - ◆ `[a-z]` matches a lowercase letter, and
 - ◆ `[a-zA-Z]` matches any lowercase or uppercase letter.

Matching Complete Strings to Patterns (Cont.)

- Custom Character Classes

- Line 21 defines a custom character class to validate a simple first name with no spaces or punctuation.
- A first name might contain many letters. In the regular expression r3 (line 21),
 - ◆ [A-Z] matches one uppercase letter, and
 - ◆ [a-z]* matches any number of lowercase letters.
 - ◆ The * quantifier matches zero or more occurrences of the subexpression to its left (in this case, [a-z]).
 - So [A-Z][a-z]* matches "Vivaan", "An" or even "E".

```
20 // match a word that starts with a capital letter
21 std::regex r3{"[A-Z][a-z]*"};
22 std::cout << "Matching against: [A-Z][a-z]*\n"
23 << std::format("Angel: {}; tina: {}\n\n",
24                 std::regex_match("Angel", r3), std::regex_match("tina", r3));
```

Matching against: [A-Z][a-z]*
Angel: true; tina: false

Matching Complete Strings to Patterns (Cont.)

- Custom Character Classes

- When a custom character class starts with a **caret (^)**, the class matches any character that's not specified.
 - ◆ So `[^a-z]` (line 27) matches any character that's not a lowercase letter.

```
26 // match any character that's not a lowercase letter
27 std::regex r4{"[^a-z]"};
28 std::cout << "Matching against: [^a-z]\n"
29 << std::format("A: {}; a: {}\n\n",
30               std::regex_match("A", r4), std::regex_match("a", r4));
```

```
Matching against: [^a-z]
A: true; a: false
```

Matching Complete Strings to Patterns (Cont.)

- Custom Character Classes

- Metacharacters in a custom character class are treated as the characters themselves. So `[*+ $]` (line 33) matches a single `*`, `+` or `$` character.

```
32 // match metacharacters as literals in a custom character class
33 std::regex r5{ "[*+ $]";
34 std::cout << "Matching against: [*+ $]\n"
35           << std::format(" *: {}; !: {}\n\n",
36                         std::regex_match(" ", r5), std::regex_match("!", r5));
```

```
Matching against: [*+ $]
 *: true; !: false
```

Matching Complete Strings to Patterns (Cont.)

- * vs. + Quantifier

- To require at least one lowercase letter in a first name, replace the * quantifier in line 21 with + (line 39).
 - ◆ This matches at least one occurrence of a subexpression to its left.

```
38 // matching a capital letter followed by at least one lowercase letter
39 std::regex r6{"[A-Z][a-z]+"};
40 std::cout << "Matching against: [A-Z][a-z]+\n"
41 << std::format("Angel: {}; T: {}\n\n",
42               std::regex_match("Angel", r6), std::regex_match("T", r6));
```

```
Matching against: [A-Z][a-z]+
Angel: true; E: false
```

Matching Complete Strings to Patterns (Cont.)

- * vs. + Quantifier

- Both * and + are **greedy**—they match as many characters as possible.
 - ◆ So the regular expression `[A-Z][a-z]+` matches any word that begins with a capital letter followed by at least one lowercase letter.
- You can make * and + **lazy**, so they match as few characters as possible by **appending a question mark (?)** to the quantifier, as in `*?` or `+?`.

Matching Complete Strings to Patterns (Cont.)

- Other Quantifiers

- The ? quantifier by itself matches **zero or one occurrence** of the subexpression to its left.
- In the regular expression `labell?ed` (line 45),
 - ◆ the subexpression is the literal character `"l"`.
 - ◆ So in the `regex_match` calls in lines 48–49, the regular expression matches `labelled` (the U.K. English spelling) and `labeled` (the U.S. English spelling),
 - ◆ but in the `regex_match` call in line 50, the regular expression does not match the misspelled word `labellled`.

```
44 // matching zero or one occurrences of a subexpression
45 std::regex r7{"labell?ed"};
46 std::cout << "Matching against: labell?ed\n"
47 << std::format("labelled: {}; labeled: {}; labellled: {}\n\n",
48                 std::regex_match("labelled", r7),
49                 std::regex_match("labeled", r7),
50                 std::regex_match("labellled", r7));
```

Matching against: labell?ed
labelled: true; labeled: true; labellled: false

Matching Complete Strings to Patterns (Cont.)

- Other Quantifiers

- You can use the `{n,}` quantifier to match at least `n` occurrences of a subexpression to its left.
- The regular expression in line 53 matches strings containing at least three digits.

```
52 // matching n (3) or more occurrences of a subexpression
53 std::regex r8{R"(\d{3,})"};
54 std::cout << R"(Matching against: \d{3,})" << "\n"
55 << std::format("123: {}; 1234567890: {}; 12: {}\n\n",
56               std::regex_match("123", r8),
57               std::regex_match("1234567890", r8),
58               std::regex_match("12", r8));
```

```
Matching against: \d{3,}
123: true; 1234567890: true; 12: false
```

Matching Complete Strings to Patterns.

- Other Quantifiers

- You can use the {n,m} quantifier to match between n and m (inclusive) occurrences of a subexpression.
- The regular expression in line 61 matches strings containing 3 to 6 digits.

```
60 // matching n to m inclusive (3-6), occurrences of a subexpression
61 std::regex r9{R"(\d{3,6})"};
62 std::cout << R"(Matching against: \d{3,6})" << "\n"
63 << std::format("123: {}; 123456: {}; 1234567: {}; 12: {}\n",
64               std::regex_match("123", r9), std::regex_match("123456", r9),
65               std::regex_match("1234567", r9), std::regex_match("12", r9));
66 }
```

```
Matching against: \d{3,6}
123: true; 123456: true; 1234567: false; 12: false
```

Replacing Substrings

- The header `<regex>` provides function `regex_replace` to replace patterns in a string.
- Let's convert a tab-delimited string to comma-delimited.

```
1 // Regular expression replacements.
2 #include <format>
3 #include <iostream>
4 #include <regex>
5 #include <string>
6
7 int main() {
8     // replace tabs with commas
9     std::string s1{"1\t2\t3\t4"};
10    std::cout << std::format("Original string: {}\n", R"(1\t2\t3\t4)")
11              << std::format("After replacing tabs with commas: {}\n",
12                             std::regex_replace(s1, std::regex{"\t"}, ","));
13 }
```

Original string: 1\t2\t3\t4

After replacing tabs with commas: 1,2,3,4

Replacing Substrings.

- The `regex_replace` (line 13) function

- receives three arguments:

- ◆ the string to be searched ("1\t2\t3\t4"),
 - ◆ the regex pattern to match (the tab character "\t") and
 - ◆ the replacement text (",").

- It returns a new string containing the modifications.

- The expression

`std::regex{"\t"}`

creates a temporary regex object, initializes it and immediately passes it into the function `regex_replace`.

- This is useful if you do not need to reuse a regex multiple times.

Searching for Matches

- You can match a substring within a string using the function `regex_search`, which returns true if any part of an arbitrary string matches the regular expression.
 - Optionally, the function also gives you access to the matching substring via an object of the class template `match_results` that you pass as an argument.
 - There are `match_results` aliases for different string types:
 - ◆ For searches in `std::strings`, use an `smatch` (pronounced “ess match”).
 - ◆ For searches on C-strings and string literals, use a `cmatch` (pronounced “see match”).
 - The `<regex>` header has **NOT** yet been updated for `string_views`.

Searching for Matches

- Finding a Match Anywhere in a String

- The calls to function `regex_search` in lines 13–15 search in `s1` ("Programming is fun") for the first occurrence of a substring that matches a regular expression—in this case, the literal strings "Programming", "fun" and "fn".
 - ◆ Function `regex_search`'s two-argument version returns true or false to indicate a match.

```
1 // Matching patterns throughout a string.
2 #include <format>
3 #include <iostream>
4 #include <regex>
5 #include <string>
6
7 int main() {
8     // performing a simple match
9     std::string s1{"Programming is fun"};
10    std::cout << std::format("s1: {}\n\n", s1);
11    std::cout << "Search anywhere in s1:\n"
12    << std::format("Programming: {}; fun: {}; fn: {}\n\n",
13                  std::regex_search(s1, std::regex{"Programming"}),
14                  std::regex_search(s1, std::regex{"fun"}),
15                  std::regex_search(s1, std::regex{"fn"}));
```

s1: Programming is fun

Search anywhere in s1:

Programming: true; fun: true; fn: false

Searching for Matches

- Ignoring Case in a Regular Expression and Viewing the Matching Text
 - The `regex_constants` in the header `<regex>` can customize how regular expressions perform matches.
 - ◆ For example, matches are case-sensitive by default.
 - ◆ Using the constant `regex_constants::icase`, you can perform a **case-insensitive search**.

```
17 // ignoring case
18 std::string s2{"AZUEL MAGAR"};
19 std::smatch match; // store the text that matches the pattern
20 std::cout << std::format("s2: {}\n\n", s2);
21 std::cout << "Case insensitive search for Azuel in s2:\n"
22 << std::format("Azuel: {}\n", std::regex_search(s2, match,
23 std::regex{"Azuel", std::regex_constants::icase}));
24 << std::format("Matched text: {}\n\n", match.str());
```

s2: AZUEL MAGAR

Case insensitive search for Azuel in s2:

Azuel: true

Matched text: AZUEL

Searching for Matches

- Ignoring Case in a Regular Expression and Viewing the Matching Text

- Lines 22–23 call `regex_search`'s three-argument version, in which the arguments are
 - ◆ the string to search (`s2`; line 22), which in this case contains all capital letters,
 - ◆ the `smatch` object (`match`; line 22), which stores the match if there is one, and
 - ◆ the regex pattern to match (line 23).

```
17 // ignoring case
18 std::string s2{"AZUEL MAGAR"};
19 std::smatch match; // store the text that matches the pattern
20 std::cout << std::format("s2: {}\n\n", s2);
21 std::cout << "Case insensitive search for Azuel in s2:\n"
22 << std::format("Azuel: {}\n", std::regex_search(s2, match,
23 std::regex{"Azuel", std::regex_constants::icase}))
24 << std::format("Matched text: {}\n\n", match.str());
```

Searching for Matches

- Ignoring Case in a Regular Expression and Viewing the Matching Text

- Here, we created the regex with a second argument `std::regex{"Azuel", std::regex_constants::icase}`
 - ◆ This regex matches the literal characters "Azuel" regardless of their case.
 - In `s2`, "AZUEL" matches the regex "Azuel" because both have the same letters, even though "AZUEL" contains only uppercase letters.
 - ◆ To confirm the matching text, line 24 gets the match's string representation by calling its member function `str`.

```
17 // ignoring case
18 std::string s2{"AZUEL MAGAR"};
19 std::smatch match; // store the text that matches the pattern
20 std::cout << std::format("s2: {}\n\n", s2);
21 std::cout << "Case insensitive search for Azuel in s2:\n"
22 << std::format("Azuel: {}\n", std::regex_search(s2, match,
23 std::regex{"Azuel", std::regex_constants::icase}))
24 << std::format("Matched text: {}\n\n", match.str());
```

Searching for Matches

- Finding All Matches in a String

- Let's extract all the U.S. phone numbers of the form ###-###-#### from a string.
- The following code finds each substring in contact (lines 27–28) that matches the regex phone (line 29) and displays the matching text.

```
26 // finding all matches
27 std::string contact{
28     "Ando Devi, Home: 555-555-1234, Work: 555-555-4321"};
29 std::regex phone{R"(\d{3}-\d{3}-\d{4})"};
30
31 std::cout << std::format("Finding phone numbers in:\n{}\n", contact);
32 while (std::regex_search(contact, match, phone)) {
33     std::cout << std::format("{}\n", match.str());
34     contact = match.suffix();
35 }
36 }
```

```
Finding phone numbers in:
Ando Devi, Home: 555-555-1234, Work: 555-555-4321
555-555-1234
555-555-4321
```

Searching for Matches.

- Finding All Matches in a String

- The while statement iterates as long as `regex_search` finds a match. Each iteration of the loop
 - ◆ displays the substring that matched the regular expression (line 33), then
 - ◆ replaces `contact` with the result of calling the match object's member function `suffix` (line 34), which returns the portion of the string that has not yet been searched.
 - This new string is used in the next call to `regex_search`.

```
31 std::cout << std::format("Finding phone numbers in:\n{}\n", contact);
32 while (std::regex_search(contact, match, phone)) {
33     std::cout << std::format("    {}\n", match.str());
34     contact = match.suffix();
35 }
36 }
```

```
Finding phone numbers in:
Ando Devi, Home: 555-555-1234, Work: 555-555-4321
    555-555-1234
    555-555-4321
```